

Associazioni tra classi

Tra i più importanti punti per la creazione di un **Software OO** consiste nel determinare come collaborino tra di loro delle classi.

Le associazioni tra classi di solito sono a coppie e possiamo trovare 4 tipologie:

-Dipendenza:

E' un'associazione che le variazioni di un elemento iniziale possono influenzare il secondo elemento. Come fornitore e cliente. Accade quando nella classe sono **presenti degli attributi di un'altra classe o in un metodo quando ha parametri definiti da un'altra classe**. Nei diagrammi UML viene rappresentata attraverso una freccia tratteggiata.

-Generalizzazione:

E' un'associazione che consiste nel fatto che permette di ereditare attributi o operazioni da una classe padre. Nel linguaggio UML viene rappresentato con una freccia continua.

-Composizione:

In questa associazione una classe (**tutto**) può contenere tutte le parti di altre classi **gestendo il loro ciclo di vita**. Questa associazione è anche detta **has-A**. Nel linguaggio UML viene rappresentato con un segmento e nella parte rivolta alla classe che "tutto" ha un rombo disegnato e durante il collegamento può essere specificata la molteplicità dell'associazione. Se nel collegamento verso la classe componente è presente una freccia indica che ci si può navigare all'interno.

-Aggregazione:

E' un'associazione meno forte della precedente, infatti può appartenere a più di una classe "tutto", indipendentemente da dove si trovi. In UML è rappresentato come l'associazione composizione ma con il rombo vuoto.

Un criterio per distinguere l'associazione di **composizione** e di **aggregazione** è verificare se l'eliminazione di un oggetto composto implica l'eliminazione dei suoi componenti: in caso affermativo si tratta di composizione, altrimenti di aggregazione.

Tipi di dato primitivi e le classi wrapper

Java è un linguaggio che quando si dichiara una variabile dobbiamo ogni volta specificare il suo tipo che può essere uno dei tipi primitivi del linguaggio(int, double, f o una classe predefinita da un utente o una libreria. Questi tipi di linguaggi si chiamano "staticamente tipizzati".

Il linguaggio java per l'uso di dati primitivi rende disponibili delle classi chiamate **wrapper** che si trovano nel package **java.lang** (non richiede importazione). Una classe wrapper trasforma, **incapsulando** il tipo di dato primitivo, in un **oggetto** a cui integra **alcune funzionalità utili**. Di solito questa classe ha lo **stesso nome del tipo primitivo** corrispondente (es. int, long, short, float, double).

La classe wrapper può avere **un solo tipo di attributo** tramite il **costruttore**, che a seguire non può essere modificato, ma acquisito invocando uno specifico metodo. Queste classi espongono molti metodi **statici di utilità** (es. convertire stringhe di caratteri in valori numerici o viceversa).

Boxing e unboxing:

- boxing avviene quando un tipo di dato primitivo viene trasformato nel suo oggetto wrapper
- unboxing quando un oggetto wrapper viene trasformato in un dato primitivo.

Nonostante queste due operazioni nelle versioni più recenti di java si è introdotto l'**autoboxing** che consiste nell'unione di **boxing e unboxing** soltanto che avviene **automaticamente**. Lo scopo principale di queste classi è fornire **metodi statici** che non **richiedono di essere applicati a oggetti**.

Le Classi Servizio forniscono servizi di utilità con metodi statici, senza costruttori.

Codifica delle stringhe

La classe **String** permette di utilizzare le stringhe in **Java** e la sintassi usata per dichiararle è **simile a quella dei tipi primitivi**. In realtà, le stringhe sono **oggetti** creati dalla classe String e memorizzati nella memoria **heap**. Nelle stringhe si possono inserire **infiniti caratteri** e per esprimere una stringa vuota si usa: **""**. Per codificare i caratteri Java piuttosto che usare **ASCII** usa **Unicode** in cui la sua codifica è basata su **16 bit** e riesce a codificare fino a **65.536 simboli diversi**.

Unione di più stringhe: L'operatore **“ + ”** serve per concatenare più stringhe che si trovano (es: **“Ciao”+“ come stai?”**).

Confronto: per confrontare più stringhe si usa l'operatore **“ == ”**

Esempio:

```
String s1= new String ("Esempio");  
s2= new String ("Esempio");  
System.out.println(s1==s2).
```

Questi oggetti del tipo **String** sono **immutabili** ovvero sono delle costanti che assumono il valore durante la loro creazione che non potrà essere modificato e se si prova a modificarlo si distrugge l'elemento precedentemente inserito.

Ultimamente grazie alla versione **Java 7** si può integrare un oggetto di tipo **String** all'interno dell'istruzione **switch case** e per far avvenire il **confronto** si usa un metodo: **equals**. Questo codice è **più efficiente** di un semplice **if-else**.

Gestioni delle date e degli orari

Un'altra funzione utile per i programmi Java è la gestione delle date che nelle versioni precedenti di Java 8 si potevano introdurre grazie a due classi: `Java.util.Date` e `java.util.Calendar`. Queste due classi però avevano dei problemi:

- Creazione di oggetti data mutabili e non costanti
- Rappresentazione di intervalli di tempo
- Inadeguatezza, salvo adeguati accorgimenti, per applicazioni concorrenti
- Mesi numerati partendo da 0
- Le classi di utilità per formattare le date utilizzabili solo con `Date` e non `Calendar`

Nella versione Java 8 si è sviluppata una nuova soluzione proprietaria in modo da offrire classi:

- Oggetti immutabili** che rappresentano un valore e non un intervallo temporale
- Thread-safe** che gestiscono automaticamente gli aspetti relativi alla concorrenza
- Caratteristiche domain-driver-design** per tener conto delle possibili situazioni in cui le date e orari vengano utilizzati e offrendo al tempo stesso un loro utilizzo immediato e comprensibile
- Disponibilità di diversi sistemi di calendario**, per supportare le esigenze di diverse zone geografiche

-Per gestire le date la classe più importante è **LocalDate** permettendo **la creazione di una data senza inserire un orario** che avrà inizio in momenti diversi a seconda di qual è la sua posizione nella terra. Quando si crea una data **partendo da valori iniziali** il nome del costruttore è **of**, invece se si **crea una data con altri dati di un'altra** il metodo usato è **form**. Infine per creare una data bisogna fornire al **costruttore** come argomento una **stringa**.

-**Java.time.temporal** è un package che permette attraverso **metodi statici** (`adjuster`) di effettuare manipolazioni standard sulle date e in caso di necessità si possono implementare dei metodi statici personalizzati.

-**LocalTime** ha le stesse caratteristiche di **LocalDate** per la gestione degli orari. Rappresenta un orario **senza nessuna data associata** ed è relativo a un **determinato fuso orario**.

-**LocalDateTime** è una classe che serve per **combinare data e ora**.

-Instant è una classe che fornisce una **precisione nell'ordine dei nanosecondi** e viene usata per la **memorizzazione e il confronto di timestamp** in cui è necessario registrare quando si è verificato un certo evento.

Java doc

Ogni volta che si crea un programma è **necessario che si documenti il codice sviluppato** e per farlo si sono creati degli **standard** per annotare il codice sorgente, soprattutto quando è molto **complesso e complicato**. Questo standard è basato sul comando **Javadoc**, una documentazione di **pagine HTML**, che genera automaticamente le documentazioni necessarie delle classi e dei relativi membri lasciando i commenti inseriti nel codice sorgente. Per poter inserire questa documentazione bisogna scrivere “ **/**** ” e premere il tasto “ **invio** ”. Una volta fatto questo procedimento lo sviluppatore ha il compito di inserire i tag, parole chiave precedute dal simbolo “ **@** ”

Procedimento per creare **Javadoc** su Eclipse:

Sul Eclipse si può generare il JavaDoc attraverso questa procedura:

Avvi eclipse>Apri progetto>Clicchi nella barra in alto “Progetto>

Genera Javadoc>Selezionare il progetto>Fine

Verrà creata una cartella doc e all'interno si può trovare la pagina html di Javadoc